

K-Nearest Neighbors (KNN): Formulas and Numerical Examples

Core Idea: KNN predicts outcomes using the k closest data points.

Distance metrics (e.g., Euclidean) identify neighbors, while prediction rules (voting for classification, averaging for regression) generate final results.

1. Key Formulas

(1) Distance Calculation: Euclidean Formula

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

Terms:

x, y : Test and training points.

x_i, y_i : Feature values (e.g., sepal length, weight).

n : Number of features.

Note:

Dominant features distort results; **standardize data first** (e.g., scale to mean=0, variance=1) .

(2) Prediction Rules

Classification (Majority Vote):

$$\hat{y} = \operatorname{argmax}_c \sum_{i=1}^k I(y_i = c)$$

$I(y_i = c)$: 1 if neighbor i has class c , else 0.

Ties: Random selection or weighted voting.

Regression (Mean Value):

$$\hat{y} = \frac{1}{k} \sum_{i=1}^k y_i$$

Weights: Optionally weight neighbors by inverse distance.

(3) Parameter Selection

k-value:

Small k (e.g., 1): High variance, noise-sensitive.

Large k (e.g., 10+): Smoothing, may miss patterns [Source 4].

Optimization: Use cross-validation to choose k .

2. Numerical Examples

(1) Euclidean Distance Calculation

Dataset: Iris (Features: sepal length, width; petal length, width) [Source 5].

Test Point: $x = (5.0, 3.3, 1.4, 0.2)$

Training Point: $y = (4.6, 3.4, 1.4, 0.3)$

Calculation:

$$d(x, y) = \sqrt{(5.0 - 4.6)^2 + (3.3 - 3.4)^2 + (1.4 - 1.4)^2 + (0.2 - 0.3)^2} = \sqrt{0.16 + 0.01 + 0 + 0.01} \\ = \sqrt{0.18} \approx 0.424$$

Key Insight: Distance is computed for all training points to find nearest neighbors.

(2) Classification Example (Majority Vote)

Task: Predict flower class ($k = 3$).

Neighbors and Classes:

Neighbor	Distance	Class
1	0.224	A
2	0.265	B
3	0.424	A

Voting:

Class A: Neighbors 1 and 3 → 2 votes.

Class B: Neighbor 2 → 1 vote.

Prediction: $\hat{y} = A$ (majority class).

(3) Regression Example (Mean Value)

Task: Predict weight (kg) for ID11 ($k = 3$).

Neighbors' Weights: 77 kg, 72 kg, 60 kg.

Calculation:

$$\hat{y} = \frac{77 + 72 + 60}{3} = \frac{209}{3} \approx 69.67kg$$

Refinement: Weighted average (closer neighbors = higher influence):

$$\hat{y}_{weighted} = \frac{(1/0.224 \times 77) + (1/0.265 \times 72) + (1/0.424 \times 60)}{1/0.224 + 1/0.265 + 1/0.424} \approx 71.2kg$$

3. Practical Considerations

1. **Feature Scaling:** Essential for accurate distances. Apply standardization:

$$x_{scaled} = \frac{x - \mu}{\sigma}$$

2. **Distance Alternatives:**

● **Manhattan:** $\sum_{i=1}^n |x_i - y_i|$ (robust to outliers).

● **Minkowski:** Generalization of Euclidean/Manhattan.

3. **Complexity:** Computationally expensive for large datasets (optimize with KD-trees).

Why KNN Matters

Strengths: No training phase, interpretable, adaptable to complex boundaries.

Weaknesses: Sensitive to irrelevant features, struggles with high-dimensional data.

Implementation Tip: Use scikit-learn (Python) or caret (R) for automatic scaling and neighbor search.